

PREDICTIVE PARSING TABLE

EX. NO. 6

Meghavarshini M

RA2211026050054

III CSE AIML B

AIM: To write a program to implement Predictive Parsing Table.

ALGORITHM:

- Start.
- Read the grammar's FIRST and FOLLOW sets.
- Initialize an empty parsing table.
- For each production in the grammar:
 1. Compute FIRST of the right-hand side (RHS).
 2. For each terminal in FIRST (excluding 'e'), add the production to the table.
 3. If 'e' is in FIRST, add the production to FOLLOW of the non-terminal.
- Print the parsing table.
- Stop.

PROGRAM:

```
#include <iostream>
```

```
#include <map>
```

```
#include <vector>
```

```
#include <set>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
map<char, map<string, string>> parsingTable;
```

```

map<char, set<string>> firstSet, followSet;

vector<pair<char, string>> productions;


void constructParsingTable() {

    for (auto &prod : productions) {

        char nonTerminal = prod.first;

        string rhs = prod.second;

        set<string> firstRHS;


        if (isupper(rhs[0])) firstRHS = firstSet[rhs[0]];

        else firstRHS.insert(string(1, rhs[0]));


        for (const string &terminal : firstRHS) {

            if (terminal != "ε") parsingTable[nonTerminal][terminal] = rhs;

        }


        if (firstRHS.count("ε")) {

            for (const string &terminal : followSet[nonTerminal]) {

                parsingTable[nonTerminal][terminal] = rhs;

            }

        }

    }

}

```

```

void printParsingTable() {
    set<string> terminals;

    for (auto &row : parsingTable) {
        for (auto &col : row.second) terminals.insert(col.first);
    }

    cout << "\nPredictive Parsing Table\n";
    cout << "-----\n";
    cout << setw(10) << "NT | ";
    for (const string &t : terminals) cout << setw(10) << t << " ";
    cout << "\n-----\n";

    for (auto &row : parsingTable) {
        cout << setw(10) << row.first << " | ";
        for (const string &t : terminals) {
            if (row.second.count(t)) cout << setw(10) << row.second[t] << " ";
            else cout << setw(10) << "-" << " ";
        }
        cout << endl;
    }
}

int main() {
    productions = { {'E', "TA"}, {'A', "+TA"}, {'A', "e"}, {'T', "FB"}, {'B', "*FB"}, {'B',

```

```
"e"}, {'F', "(E)"}, {'F', "id"} };
```

```
firstSet['E'] = {"(", "id"};
```

```
firstSet['A'] = {"+", "e"};
```

```
firstSet['T'] = {"(", "id"};
```

```
firstSet['B'] = {"*", "e"};
```

```
firstSet['F'] = {"(", "id"};
```

```
followSet['E'] = {"$", ")"};
```

```
followSet['A'] = {"$", ")"};
```

```
followSet['T'] = {"+", "$", ")"};
```

```
followSet['B'] = {"+", "$", ")"};
```

```
followSet['F'] = {"*", "+", "$", ")"};
```

```
constructParsingTable();
```

```
printParsingTable();
```

```
return 0;
```

```
}
```

INPUT(STRING):

```
productions = { {'E', "TA"}, {'A', "+TA"}, {'A', "e"}, {'T', "FB"}, {'B', "*FB"}, {'B', "e"},
```

```
{'F', "(E)"}, {'F', "id"} };
```

```
firstSet['E'] = {"(", "id"};
```

```
firstSet['A'] = {"+", "e"};
```

```
firstSet['T'] = {"(", "id"};
```

```
firstSet['B'] = {"*", "e"};
```

```
firstSet['F'] = {"(", "id"};
```

```
followSet['E'] = {"$", ")"};
```

```
followSet['A'] = {"$", ")"};
```

```
followSet['T'] = {"+", "$", ")"};
```

```
followSet['B'] = {"+", "$", ")"};
```

```
followSet['F'] = {"*", "+", "$", ")"};
```

OUTPUT:

Predictive Parsing Table							

NT	(	id	+	*	\$	)	

E	TA	TA	-	-	-	-	
A	-	-	+TA	-	"e"	"e"	
T	FB	FB	-	-	-	-	
B	-	-	-	*FB	"e"	"e"	
F	(E)	id	-	-	-	-	